

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и технологий

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Группа 5130201/30101

Практическая работа №3

Вариант 10

Дисциплина «Сети ЭВМ и телекоммуникации
компьютерных сетей»

Выполнил студент

Мелещенко С. И.

Преподаватель

Мулюха В.А.

«_____» _____ 2026г.

Санкт-Петербург, весна 2026

1 Постановка задачи

Используя Docker, создать контейнеры, необходимые для реализации следующего функционала с использованием RabbitMQ, а также показать, как именно осуществляется передача в этих условиях.

Вариант 10: Организовать отправку сообщений от двух отправителей одному получателю. Получатель читает сообщения значительно реже их отправки, что приводит к переполнению очереди. Задать различные стратегии переполнения очереди (drop-head, reject-publish, reject-publish-dlx) и продемонстрировать разницу в поведении.

2 Описание реализации

Был установлен Docker Desktop для Windows. После установки запущен демон Docker.

2.1 Структура проекта

```
nets3/  
  docker-compose.yml  
  producer/  
    Dockerfile  
    producer.py  
  consumer/  
    Dockerfile  
    consumer.py
```

2.2 Docker Compose

Файл docker-compose.yml используется Docker Compose для определения и запуска нескольких контейнеров как единого приложения. В данной работе он описывает четыре сервиса: брокер сообщений RabbitMQ, два сервиса-отправителя (продюсера) и один сервис-получатель (консьюмер). Все контейнеры работают в одной изолированной сети и взаимодействуют через RabbitMQ.

Листинг 1. docker-compose.yml

```
1 services:  
2   rabbitmq: # сервис брокера  
3     image: rabbitmq:3.13-management-alpine  
4     container_name: rabbitmq  
5     ports:  
6       - "5672:5672"  
7       - "15672:15672"  
8     environment:  
9       RABBITMQ_DEFAULT_USER: Sofia  
10      RABBITMQ_DEFAULT_PASS: 123  
11     healthcheck:  
12       test: ["CMD", "rabbitmq-diagnostics", "ping"]  
13       interval: 10s  
14       timeout: 5s  
15       retries: 5  
16  
17   publisher1: # первый отправитель  
18     build: ./producer  
19     environment:  
20       PUBLISHER_ID: 1  
21       AMQP_URL: amqp://Sofia:123@rabbitmq:5672/%2F
```

```

22     QUEUE_NAME: overflow_queue
23     OVERFLOW_STRATEGY: ${OVERFLOW_STRATEGY:-drop-head}
24     SEND_INTERVAL: 0.1
25     depends_on:
26         rabbitmq:
27             condition: service_healthy
28
29     publisher2: # второй отправитель
30         build: ./producer
31         environment:
32             PUBLISHER_ID: 2
33             AMQP_URL: amqp://Sofia:123@rabbitmq:5672/%2F
34             QUEUE_NAME: overflow_queue
35             OVERFLOW_STRATEGY: ${OVERFLOW_STRATEGY:-drop-head}
36             SEND_INTERVAL: 0.1
37         depends_on:
38             rabbitmq:
39                 condition: service_healthy
40
41     consumer: # получатель
42         build: ./consumer
43         environment:
44             AMQP_URL: amqp://Sofia:123@rabbitmq:5672/%2F
45             QUEUE_NAME: overflow_queue
46             CONSUME_INTERVAL: 5
47             OVERFLOW_STRATEGY: ${OVERFLOW_STRATEGY:-drop-head}
48         depends_on:
49             rabbitmq:
50                 condition: service_healthy

```

1. image rabbitmq:3.13-management-alpine - Используется официальный образ RabbitMQ версии 3.13 с включённым веб-интерфейсом управления (management) на базе Alpine Linux (лёгкий дистрибутив).

2. container_name rabbitmq - Имя контейнера, по которому к нему могут обращаться другие контейнеры (как DNS-имя).

3. ports

"5672:5672" Проброс порта 5672 (AMQP) с хоста в контейнер. Через этот порт подключаются продюсеры и консьюмер.

"15672:15672" Проброс порта 15672 – веб-интерфейс управления RabbitMQ (HTTP). Доступен на хосте по адресу <http://localhost:15672>.

4. environment

RABBITMQ_DEFAULT_USER: Sofia - Устанавливает имя пользователя по умолчанию для доступа к RabbitMQ (логин для веб-интерфейса и AMQP).

RABBITMQ_DEFAULT_PASS: 123 - Пароль для этого пользователя.

5. healthcheck

test: ["CMD "rabbitmq-diagnostics "ping"] – Проверка готовности сервиса: выполняется команда rabbitmq-diagnostics ping. Если она успешна, контейнер считается здоровым.

interval: 10s - Проверка каждые 10 секунд.

timeout: 5s - Таймаут выполнения проверки – 5 секунд.

retries: 5 - Количество неудачных попыток до признания контейнера нездоровым.

6. build: ./producer - Указывает Docker Compose собрать образ для этого сервиса из Dockerfile, находящегося в папке ./producer.

7. environment - Переменные окружения, передаваемые внутрь контейнера.

8. PUBLISHER_ID: 1 Идентификатор отправителя (1 или 2). Используется в коде для различия логов.

9. AMQP_URL: amqp://Sofia:123@rabbitmq:5672/%2F- Строка подключения к RabbitMQ. Содержит протокол amqp, логин Sofia, пароль 123, хост rabbitmq (имя контейнера), порт 5672. %2F – закодированный символ / (виртуальный хост по умолчанию).

10. QUEUE_NAME: overflow_queue - Имя очереди, в которую будут отправляться сообщения.

11. OVERFLOW_STRATEGY: \$OVERFLOW_STRATEGY:-drop-head - Ключевая переменная. Задаёт стратегию переполнения очереди. Значение берётся из переменной окружения OVERFLOW_STRATEGY операционной системы. Если она не задана, используется значение по умолчанию drop-head. Это позволяет легко переключать стратегии без правки файла.

12. SEND_INTERVAL: 0.1 - Интервал между отправками сообщений (в секундах). 0.1 с = 10 сообщений в секунду.

13. depends_on - Указывает, что сервис должен запускаться после RabbitMQ, но не ждёт его полной готовности (только запуска). Однако с condition: service_healthy Compose ожидает, пока RabbitMQ не пройдёт healthcheck (станет здоровым). Это предотвращает ошибки подключения на старте.

14. build: ./consumer - Сборка образа из Dockerfile в папке ./consumer.

15. CONSUME_INTERVAL: 5 - Время обработки одного сообщения (в секундах). Консьюмер «спит» 5 секунд перед подтверждением, имитируя медленную обработку.

2.3 Dockerfile

Для продюсера и консьюмера используются одинаковые Dockerfile:

Листинг 2. Dockerfile для producer и consumer

```
1 FROM python:3.10-slim
2 WORKDIR /app
3 RUN pip install pika
4 COPY python/producer.py .
5 CMD ["python", "producer.py"] # или consumer.py
```

1. FROM python:3.10-slim - задаёт базовый образ, на основе которого строится новый образ. python:3.10-slim – официальный образ Python версии 3.10 из Docker Hub. Суффикс slim означает, что образ содержит только минимально необходимые компоненты для работы Python.

2. WORKDIR /app - создаёт внутри контейнера директорию /app и делает её текущей рабочей директорией для всех последующих инструкций (RUN, COPY, CMD).

3. RUN pip install pika - выполняет команду внутри контейнера на этапе сборки образа. Устанавливает Python-библиотеку Pika – клиент для RabbitMQ (реализует протокол AMQP 0-9-1). Без этой библиотеки наш продюсер (и консьюмер) не смог бы подключаться к брокеру.

4. COPY . . - копирует файл с хост-машины (из той же директории, где находится Dockerfile) в текущую рабочую директорию контейнера (/app). Точка (.) означает «в текущую рабочую директорию», которая установлена WORKDIR /app. В результате внутри контейнера появляется файл /app/producer.py, который содержит код отправителя сообщений.

5. CMD ["python "producer.py"] - задаёт команду, которая будет выполнена при запуске контейнера (не при сборке). Запускает интерпретатор Python с аргументом producer.py, то есть выполняет код продюсера. Используется форма exes (список строк) – это предпочтительный способ, так как сигналы (например, Ctrl+C для остановки) правильно доставляются процессу Python.

3 Реализация процессов

3.1 Код продюсера (producer.py)

Продюсер при старте считывает из переменных окружения свой идентификатор, имя очереди, интервал отправки, стратегию переполнения и URL подключения к RabbitMQ, после чего подключается к брокеру с повторными попытками (до 10 раз с паузой 5 секунд). Затем он создаёт канал и формирует аргументы очереди с ограничением длины в 10 сообщений и заданной стратегией (x-overflow); если выбрана стратегия reject-publish-dlx, дополнительно объявляется обмен dlx_exchange, очередь dead_letter_queue и привязка к этому обмену, а в аргументы очереди добавляются x-dead-letter-exchange и x-dead-letter-routing-key. Для стратегии reject-publish включается режим подтверждений (confirm_delivery), чтобы отслеживать отклонения сообщений. Далее в бесконечном цикле продюсер увеличивает счётчик, формирует сообщение вида Msg X from pubY и публикует его в очередь через basic_publish с пустым обменом; параметр mandatory выставляется только для reject-publish. При стратегии reject-publish возможное исключение NackError перехватывается и логируется как REJECTED (queue full), остальные исключения также обрабатываются. После каждой отправки программа делает паузу 0.1 секунды. Остановка по Ctrl+C закрывает соединение.

Листинг 3. producer.py

```
1 import os
2 import time
```

```

3 import pika
4
5 PUBLISHER_ID = os.environ.get("PUBLISHER_ID", "0")
6 QUEUE_NAME = os.environ.get("QUEUE_NAME", "overflow_queue")
7 SEND_INTERVAL = float(os.environ.get("SEND_INTERVAL", "0.1"))
8 OVERFLOW_STRATEGY = os.environ.get("OVERFLOW_STRATEGY", "drop-
  head")
9 AMQP_URL = os.environ.get("AMQP_URL", "amqp://Sofia:123@rabbitmq
  :5672/%2F")
10
11 def log(msg):
12     print(f"[Publisher {PUBLISHER_ID}] {msg}", flush=True)
13
14 def connect_to_rabbitmq(retries=10, delay=5):
15     params = pika.URLParameters(AMQP_URL)
16     for attempt in range(retries):
17         try:
18             return pika.BlockingConnection(params)
19         except pika.exceptions.AMQPConnectionError:
20             log(f"RabbitMQ not ready, retrying in {delay}
21                 seconds (attempt {attempt+1}/{retries})...")
22             time.sleep(delay)
23     raise Exception("Could not connect to RabbitMQ")
24
25 def main():
26     log(f"Starting. Send interval = {SEND_INTERVAL}s, overflow={
27         OVERFLOW_STRATEGY}")
28     connection = connect_to_rabbitmq()
29     channel = connection.channel()
30
31     # Аргументы очереди
32     arguments = {"x-max-length": 10, "x-overflow":
33         OVERFLOW_STRATEGY}
34     if OVERFLOW_STRATEGY == "reject-publish-dlx":
35         channel.exchange_declare(exchange="dlx_exchange",
36             exchange_type="direct", durable=True)
37         channel.queue_declare(queue="dead_letter_queue", durable
38             =True)
39         channel.queue_bind(queue="dead_letter_queue", exchange="
40             dlx_exchange", routing_key="")
41         arguments["x-dead-letter-exchange"] = "dlx_exchange"
42         arguments["x-dead-letter-routing-key"] = ""
43
44     channel.queue_declare(queue=QUEUE_NAME, durable=True,
45         arguments=arguments)
46
47     # Включаем подтверждения для reject-publish
48     if OVERFLOW_STRATEGY == "reject-publish":

```

```

42         channel.confirm_delivery()
43
44     counter = 0
45     try:
46         while True:
47             counter += 1
48             msg = f"Msg {counter} from pub{PUBLISHER_ID}"
49             try:
50                 channel.basic_publish(
51                     exchange='',
52                     routing_key=QUEUE_NAME,
53                     body=msg,
54                     mandatory=(OVERFLOW_STRATEGY == "reject-publish"), # mandatory нужен только для
55                                     reject-publish
56                                     properties=pika.BasicProperties(
57                                         delivery_mode=2)
58                                     )
59                 log(f"Sent: {msg}")
60             except pika.exceptions.NackError:
61                 log(f"REJECTED (queue full): {msg}")
62             except pika.exceptions.UnroutableError:
63                 log(f"UNROUTABLE: {msg}")
64             time.sleep(SEND_INTERVAL)
65         except KeyboardInterrupt:
66             log("Stopped")
67             connection.close()
68
69 if __name__ == "__main__":
70     main()

```

3.2 Код консьюмера (consumer.py)

Консьюмер при старте считывает из переменных окружения имя очереди, интервал обработки, стратегию переполнения и URL подключения к RabbitMQ, после чего подключается к брокеру с повторными попытками (до 10 раз с паузой 5 секунд). Затем он создаёт канал и формирует аргументы очереди с ограничением длины в 10 сообщений и заданной стратегией; если выбрана стратегия reject-publish-dlx, дополнительно объявляется обмен dlx_exchange, создаётся очередь dead_letter_queue и выполняется её привязка к этому обмену, а в аргументы основной очереди добавляются x-dead-letter-exchange и x-dead-letter-routing-key. После объявления очереди консьюмер устанавливает QoS с prefetch_count=1, чтобы одновременно обрабатывать не более одного сообщения. Далее определяется callback-функция, которая декодирует полученное сообщение, логирует его, имитирует обработку с помощью time.sleep(CONSUME_INTERVAL) (5 секунд по умолчанию) и затем отправляет брокеру подтверждение basic_ack. Консьюмер подписывается на очередь с auto_ack=False, чтобы сообщение не удалялось до явного подтверждения, и запус-

кает цикл приёма сообщений (start_consuming). При нажатии Ctrl+C цикл прерывается, потребление останавливается и соединение закрывается.

Листинг 4. consumer.py

```
1 import os
2 import time
3 import pika
4
5 QUEUE_NAME = os.environ.get("QUEUE_NAME", "overflow_queue")
6 CONSUME_INTERVAL = int(os.environ.get("CONSUME_INTERVAL", "5"))
7 OVERFLOW_STRATEGY = os.environ.get("OVERFLOW_STRATEGY", "drop-
  head")
8 AMQP_URL = os.environ.get("AMQP_URL", "amqp://user:
  password@rabbitmq:5672/%2F")
9
10 def log(msg):
11     print(f"[Consumer] {msg}", flush=True)
12
13 def connect_to_rabbitmq(retries=10, delay=5):
14     params = pika.URLParameters(AMQP_URL)
15     for attempt in range(retries):
16         try:
17             return pika.BlockingConnection(params)
18         except pika.exceptions.AMQPConnectionError:
19             log(f"RabbitMQ not ready, retrying in {attempt+1}/{
20                 retries}...")
21             time.sleep(delay)
22     raise Exception("Could not connect to RabbitMQ")
23
24 def main():
25     log(f"Starting. Consume interval = {CONSUME_INTERVAL}s")
26     connection = connect_to_rabbitmq()
27     channel = connection.channel()
28
29     arguments = {"x-max-length": 10, "x-overflow":
30                 OVERFLOW_STRATEGY}
31     if OVERFLOW_STRATEGY == "reject-publish-dlx":
32         channel.exchange_declare(exchange="dlx_exchange",
33                                 exchange_type="direct", durable=True)
34         # Создаём очередь для мёртвых сообщений и привязываем к
35         # DLX
36         channel.queue_declare(queue="dead_letter_queue", durable
37                               =True)
38         channel.queue_bind(queue="dead_letter_queue", exchange="
39                             dlx_exchange", routing_key="")
40         arguments["x-dead-letter-exchange"] = "dlx_exchange"
41         arguments["x-dead-letter-routing-key"] = ""
```

```

37     channel.queue_declare(queue=QUEUE_NAME, durable=True,
38                             arguments=arguments)
39     channel.basic_qos(prefetch_count=1)
40
41     def callback(ch, method, properties, body):
42         msg = body.decode()
43         log(f"Received: {msg}")
44         time.sleep(CONSUME_INTERVAL)
45         ch.basic_ack(delivery_tag=method.delivery_tag)
46         log(f"Finished: {msg}")
47
48     channel.basic_consume(queue=QUEUE_NAME, on_message_callback=
49                             callback, auto_ack=False)
50     log("Waiting for messages...")
51     try:
52         channel.start_consuming()
53     except KeyboardInterrupt:
54         channel.stop_consuming()
55         connection.close()
56
57 if __name__ == "__main__":
58     main()

```

4 Результаты работы

Запуск системы производился командой:

```
docker-compose down -v
$env:OVERFLOW_STRATEGY="drop-head"    # или reject-publish / reject-publish-dlx
docker-compose up --build
```

4.1 Стратегия drop-head

При этой стратегии продюсеры отправляют сообщения без ошибок.

Листинг 5. Фрагмент лога консьюмера (drop-head)

```
1 publisher1 -1 | [Publisher 1] Sent: Msg 2 from pub1
2 publisher2 -1 | [Publisher 2] Sent: Msg 2 from pub2
3 publisher1 -1 | [Publisher 1] Sent: Msg 3 from pub1
4 publisher2 -1 | [Publisher 2] Sent: Msg 3 from pub2
5 publisher1 -1 | [Publisher 1] Sent: Msg 4 from pub1
6 publisher2 -1 | [Publisher 2] Sent: Msg 4 from pub2
7 publisher1 -1 | [Publisher 1] Sent: Msg 5 from pub1
8 publisher2 -1 | [Publisher 2] Sent: Msg 6 from pub2
```

Консьюмер обрабатывает сообщения раз в 5 секунд. Из-за ограничения очереди (10 сообщений) старые сообщения удаляются, и консьюмер получает только самые свежие. Например, в логах видно получение сообщений с пропусками: Msg 1, затем Msg 41, Msg 85 и т.д.

Листинг 6. Фрагмент лога консьюмера (drop-head)

```
1 consumer-1 | [Consumer] Received: Msg 1 from pub1
2 consumer-1 | [Consumer] Finished: Msg 1 from pub1
3 consumer-1 | [Consumer] Received: Msg 41 from pub1
4 consumer-1 | [Consumer] Finished: Msg 41 from pub1
5 consumer-1 | [Consumer] Received: Msg 85 from pub1
```

Overview					Messages			Message rates			+
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	overflow_queue	classic	D Lim Ovfl	running	10	1	11	20/s	0.20/s	0.20/s	

Рис. 1. Queue

4.2 Стратегия reject-publish

При переполнении очереди продюсеры начинают получать исключение NackError, которое логируется как «REJECTED». После того как консьюмер освобождает место, следующее сообщение успешно принимается, затем очередь снова переполняется.

Листинг 7. Фрагмент лога продюсера (reject-publish)

```

1 publisher2 -1 | [Publisher 2] Sent: Msg 1 from pub2
2 publisher1 -1 | [Publisher 1] Sent: Msg 1 from pub1
3 publisher2 -1 | [Publisher 2] Sent: Msg 2 from pub2
4 publisher1 -1 | [Publisher 1] Sent: Msg 2 from pub1
5 publisher2 -1 | [Publisher 2] Sent: Msg 3 from pub2
6 publisher1 -1 | [Publisher 1] Sent: Msg 3 from pub1
7 publisher2 -1 | [Publisher 2] Sent: Msg 4 from pub2
8 publisher1 -1 | [Publisher 1] Sent: Msg 4 from pub1
9 publisher2 -1 | [Publisher 2] Sent: Msg 5 from pub2
10 publisher1 -1 | [Publisher 1] Sent: Msg 5 from pub1
11 publisher1 -1 | [Publisher 1] REJECTED (queue full): Msg 6 from
    pub
12 ...
13 consumer-1 | [Consumer] Finished: Msg 1 from pub2
14 consumer-1 | [Consumer] Received: Msg 1 from pub1
15 publisher1 -1 | [Publisher 1] REJECTED (queue full): Msg 50 from
    pub1
16 publisher2 -1 | [Publisher 2] Sent: Msg 50 from pub2
17 publisher2 -1 | [Publisher 2] REJECTED (queue full): Msg 51 from
    pub2
18 publisher1 -1 | [Publisher 1] REJECTED (queue full): Msg 51 from
    pub1

```

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	overflow_queue	classic	D Lim Ovfl	running	10	1	11	38/s	0.40/s	0.40/s	

Рис. 2. Queue

4.3 Стратегия reject-publish-dlx

Продюсеры не получают ошибок, все сообщения успешно отправляются.

Листинг 8. Фрагмент лога продюсера (reject-publish-dlx)

```

1 publisher1 -1 | [Publisher 1] Sent: Msg 1 from pub1
2 publisher2 -1 | [Publisher 2] Sent: Msg 1 from pub2
3 consumer-1 | [Consumer] Waiting for messages ...
4 consumer-1 | [Consumer] Received: Msg 1 from pub1
5 publisher2 -1 | [Publisher 2] Sent: Msg 2 from pub2
6 publisher1 -1 | [Publisher 1] Sent: Msg 2 from pub1
7 publisher1 -1 | [Publisher 1] Sent: Msg 3 from pub1
8 publisher2 -1 | [Publisher 2] Sent: Msg 4 from pub2
9 publisher1 -1 | [Publisher 1] Sent: Msg 4 from pub1
10 publisher2 -1 | [Publisher 2] Sent: Msg 5 from pub2

```

```

11 publisher1 -1 | [Publisher 1] Sent: Msg 5 from pub1
12 publisher1 -1 | [Publisher 1] Sent: Msg 6 from pub1
13 publisher2 -1 | [Publisher 2] Sent: Msg 6 from pub2

```

При переполнении очереди лишние сообщения автоматически перенаправляются в Dead Letter Queue `dead_letter_queue`. В веб-интерфейсе RabbitMQ видны две очереди: основная с 10 сообщениями и очередь DLX с накопленными сообщениями.

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	dead_letter_queue	classic	D	running	1,375	0	1,375				
/	overflow_queue	classic	D Lim Ovfl DLX DLK	running	10	1	11	20/s	0.00/s	0.00/s	

Рис. 3. Queue

4.4 Сравнение стратегий

Стратегия	Ошибки у продюсера	Потеря сообщений	Наличие DLQ
drop-head	нет	да (старые)	нет
reject-publish	да (REJECTED)	нет (отказ)	нет
reject-publish-dlx	нет	нет	да

Таблица 1

Сравнение поведения трёх стратегий переполнения

5 Заключение

В ходе лабораторной работы были созданы Docker-контейнеры с RabbitMQ, двумя продюсерами и одним консьюмером. Реализована очередь с ограничением длины 10 сообщений. Продемонстрированы три стратегии управления переполнением: drop-head (удаление старых), reject-publish (отказ новых) и reject-publish-dlx (перенаправление в Dead Letter Queue). С помощью логов и веб-интерфейса показана разница в поведении каждой стратегии. Контейнеры успешно взаимодействуют через Docker Compose.